

管理员系统 — 完整技术方案

一、涉及文件清单

新增 5 个文件：

```
src/main/java/com/example/chat/admin/
├─ AdminResource.java      REST API (约 130 行)
├─ AdminService.java       业务逻辑 + 权限校验 (约 90 行)
├─ AdminDao.java           管理员专属 SQL (约 100 行)
├─ DashboardStats.java     record (1 行)
```

```
src/main/webapp/assets/js/
├─ admin.js                管理面板前端 (~200 行)
```

修改 8 个文件：

SessionSupport.java	+ requireAdmin()
User.java	+ disabled 字段
UserDao.java	+ disabled 查询支持
UserService.java	+ listAll() / setDisabled() / setRole()
SchemaMigrator.java	+ disabled 列 + audit_logs 表
schema.sql	+ 同上
app.html	+ adminView 面板 + adminNavButton
app.js	+ admin 按钮显隐逻辑

二、数据库变更

```
-- users 表新增
ALTER TABLE users ADD COLUMN disabled TINYINT(1) NOT NULL DEFAULT 0;

-- 操作日志表
CREATE TABLE IF NOT EXISTS admin_audit_logs (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  admin_id BIGINT NOT NULL,
  action VARCHAR(40) NOT NULL,
  target_type VARCHAR(40),
  target_id BIGINT,
  detail VARCHAR(500),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (admin_id) REFERENCES users(id)
);
```

SchemaMigrator 新增两个方法：`ensureUsersDisabledColumn()` 和 `ensureAdminAuditLogsTable()`。

三、权限拦截

`SessionSupport.java` 新增:

```
public static long requireAdmin(HttpServletRequest request) {
    long userId = requireUserId(request);
    try (Connection c = Database.connection()) {
        User user = new UserService().get(userId);
        if (!"ADMIN".equals(user.role())) {
            throw new AppException(Response.Status.FORBIDDEN, "需要管理员权限");
        }
        if (user.disabled()) {
            throw new AppException(Response.Status.FORBIDDEN, "账号已被禁用");
        }
    } catch (AppException e) { throw e; }
    catch (Exception e) { throw AppException.badRequest("鉴权失败"); }
    return userId;
}
```

每个 Admin API 方法开头调用 `requireAdmin()`。

四、后端 API

所有端点前缀: `/api/admin`

4.1 仪表盘

GET `/api/admin/dashboard`

```
{
  "ok": true,
  "data": {
    "totalUsers": 128,
    "totalMessages": 18420,
    "totalGroups": 15,
    "totalMoments": 89,
    "activeUsersToday": 42,
    "newUsersThisWeek": 23,
    "messagesToday": 1203,
    "onlineUsers": 7
  }
}
```

```
// AdminDao.java - 一条 SQL 搞定
public DashboardStats dashboard(Connection c) throws SQLException {
```

```

return Jdbc.one(c, ""
    SELECT
        (SELECT COUNT(*) FROM users) total_users,
        (SELECT COUNT(*) FROM messages) total_messages,
        (SELECT COUNT(*) FROM chat_groups) total_groups,
        (SELECT COUNT(*) FROM moments WHERE deleted=0) total_moments,
        (SELECT COUNT(DISTINCT sender_id) FROM messages
         WHERE sent_at >= CURDATE()) active_users_today,
        (SELECT COUNT(*) FROM users
         WHERE created_at >= DATE_SUB(CURDATE(), INTERVAL 7 DAY))
new_users_this_week,
        (SELECT COUNT(*) FROM messages
         WHERE sent_at >= CURDATE()) messages_today
    "", ps -> {},
rs -> new DashboardStats(
    rs.getLong("total_users"), rs.getLong("total_messages"),
    rs.getLong("total_groups"), rs.getLong("total_moments"),
    rs.getLong("active_users_today"), rs.getLong("new_users_this_week"),
    rs.getLong("messages_today")));
}

```

4.2 用户管理

GET	/api/admin/users?q=&page=1&size=20	→ 用户列表（分页 + 搜索）	
PUT	/api/admin/users/{id}/role	→ 设为/撤销管理员	body: {"role": "ADMIN"}
PUT	/api/admin/users/{id}/disable	→ 禁用用户	body: {"disabled": true}
DELETE	/api/admin/users/{id}	→ 永久删除用户	

```

// AdminDao.java - 用户列表
public List<User> listUsers(Connection c, String q, int offset, int limit) throws
SQLException {
    String like = "%" + (q == null ? "" : q.trim()) + "%";
    return Jdbc.list(c,
        "SELECT
id,username,qq_email,nickname,avatar_url,background_url,signature,role,disabled "
+
        "FROM users WHERE username LIKE ? OR qq_email LIKE ? OR nickname LIKE ? "
+
        "ORDER BY id LIMIT ? OFFSET ?",
ps -> { ps.setString(1, like); ps.setString(2, like); ps.setString(3,
like);
        ps.setInt(4, limit); ps.setInt(5, offset); },
this::mapUser);
}

public int countUsers(Connection c, String q) throws SQLException { ... }

```

```

public void setDisabled(Connection c, long userId, boolean disabled) throws
SQLException {
    Jdbc.update(c, "UPDATE users SET disabled=? WHERE id=?", ps -> {
        ps.setBoolean(1, disabled); ps.setLong(2, userId); });
}

public void setRole(Connection c, long userId, String role) throws SQLException {
    Jdbc.update(c, "UPDATE users SET role=? WHERE id=?", ps -> {
        ps.setString(1, role); ps.setLong(2, userId); });
}

public void deleteUser(Connection c, long userId) throws SQLException {
    Jdbc.update(c, "DELETE FROM users WHERE id=? AND role<>'ADMIN'", ps ->
ps.setLong(1, userId));
    // 管理员不能被删除
}

```

User record 新增字段:

```

public record User(..., boolean disabled) {}

```

禁止被禁用的用户登录: 在 `AuthService.login()` 中 `findByUsername()` 之后检查 `user.disabled()`。被禁用者即使密码正确也收到 "账号已被禁用" 的错误。

4.3 群组管理

GET	/api/admin/groups?page=1&size=20	→ 全部群聊列表 (含成员数 + 消息数)
DELETE	/api/admin/groups/{conversationId}	→ 解散群聊

```

// AdminDao.java
public List<GroupSummary> listGroups(Connection c, int offset, int limit) throws
SQLException {
    return Jdbc.list(c, ""
        SELECT cg.id, cg.name, cg.conversation_id, u.nickname owner_name,
            (SELECT COUNT(*) FROM chat_group_members WHERE group_id=cg.id)
member_count,
            (SELECT COUNT(*) FROM messages WHERE
conversation_id=cg.conversation_id) message_count,
            cg.created_at
        FROM chat_groups cg JOIN users u ON u.id=cg.owner_id
        ORDER BY cg.created_at DESC LIMIT ? OFFSET ?
        "", ps -> { ps.setInt(1, limit); ps.setInt(2, offset); },
        rs -> new GroupSummary(...));
}

```

解散群聊：删除 `conversation_members`、`chat_group_members`、`chat_group_invitations`、`messages`（通过 `message_visibility` 软删）、`chat_groups`、`conversations`。所有硬删除。

4.4 朋友圈审核

GET `/api/admin/moments?page=1&size=20` → 全部朋友圈（无视可见性，含已删除）
DELETE `/api/admin/moments/{id}` → 删除违规朋友圈（永久删除）

```
// AdminDao.java
public List<MomentView> listAllMoments(Connection c, int offset, int limit) throws
SQLException {
    // SELECT * FROM moments JOIN users - 不加 deleted=0 条件，管理员看到全部
}
```

4.5 系统公告

POST `/api/admin/announcements` → 发送全站公告

```
// AdminService.java
public void sendAnnouncement(long adminId, String content) {
    Transactional.withConnection(c -> {
        List<Long> allUserIds = adminDao.allUserIds(c);
        for (Long userId : allUserIds) {
            // 创建一条 SYSTEM 类型的私聊消息到自己——最简单但有效的推送方式
            // 或者：写入一张 announcement 表，前端 boot 时轮询
        }
        adminDao.logAction(c, adminId, "SEND_ANNOUNCEMENT", "SYSTEM", 0, content);
    });
}
```

简化方案：公告写入 `admin_announcements` 表，前端 boot 时 GET `/api/admin/announcements` 获取最新公告弹窗展示。

4.6 操作日志

GET `/api/admin/audit-logs?page=1&size=30` → 操作日志列表

每行显示：时间 | 管理员 | 操作类型 | 目标 | 详情

```
// AdminDao.java - 写日志（在 AdminService 的每个管理操作中调用）
public void logAction(Connection c, long adminId, String action,
```

```
String targetType, long targetId, String detail) throws  
SQLException {  
    Jdbc.update(c, "INSERT INTO  
admin_audit_logs(admin_id,action,target_type,target_id,detail) VALUES(?,?,?,?,?)",  
        ps -> { ps.setLong(1, adminId); ps.setString(2, action);  
                ps.setString(3, targetType); ps.setLong(4, targetId);  
ps.setString(5, detail); });  
}
```

五、前端设计

5.1 管理入口

app.html 导航栏新增：

```
<button class="rail-button" data-view="adminView" id="adminNavButton" hidden>  
    <i data-lucide="shield"></i><span>管理</span>  
</button>
```

app.js boot 时：

```
if (AppState.me && AppState.me.role === "ADMIN") {  
    $("#adminNavButton").hidden = false;  
}
```

5.2 管理面板布局



6 个标签页 data-admin-tab，点击切换。admin.js 管理所有标签页的渲染。

5.3 用户管理表格

搜索： [] [搜索]

ID	头像	用户名	QQ邮箱	角色	状态	操作
1		alice	10001@qq	USER	正常	[设管理] [禁用] [删除]
3		admin	10000@qq	ADMIN	正常	[撤管理]

第 1/7 页 [上一页] [下一页]

5.4 群组管理表格

ID	群名	群主	成员数	消息数	操作
2	Demo Grp	admin	3	156	[查看] [解散]

5.5 操作日志表格

时间	管理员	操作	详情
2026-06-01 22:30	admin	DISABLE_USER	禁用 alice(ID:1)
2026-06-01 22:28	admin	DELETE_MOMENT	删除动态(ID:5)
2026-06-01 22:25	admin	DISBAND_GROUP	解散群组(ID:2)

六、权限矩阵

操作	USER	ADMIN
使用聊天/朋友圈/语音	✓	✓
查看管理面板	✗	✓
查看全站仪表盘	✗	✓
搜索/列出所有用户	✗	✓
设置/撤销管理员角色	✗	✓
禁用/解禁用户	✗	✓
删除普通用户	✗	✓
查看所有群聊	✗	✓

解散任何群聊	✗	✓
查看所有朋友圈（含已删）	✗	✓
删除任何朋友圈	✗	✓
发送全站公告	✗	✓
查看操作日志	✗	✓

七、实施步骤

步骤	内容	时间
1	User.java 加 disabled + SchemaMigrator 加列	20m
2	SessionSupport.requireAdmin()	10m
3	AdminDao + AdminService + AdminResource（全部 API）	2h
4	UserDao.userById() 支持查 disabled + AuthService 登录拦截禁用用户	20m
5	admin.js + adminView HTML + app.js 按钮显隐	2h
6	测试：admin 登录验证所有功能	30m
合计		~5 小时